esprit

Se former autrement

2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : Arctic

Design and Implementation of a Cloud-Native Open-Source SOC Platform

By: *Ismail Triki*

Academic supervisor: *Ramzi Ouafi*

Corporate Internship Supervisor: *Hossem Mannai*

flexos TUNISIE

Network & Security Services

unesco unitwin

cdio

European Accreditation of Engineering Programmes EUR-ACE®

Délivrée par: Cti

Conférence des Grandes Écoles

Excellence for a complex world

To my parents, whose sacrifices and unwavering support
made every line of code and every sleepless night worthwhile.

To my professors at ESPRIT, who guided me
with patience and expertise throughout this journey.

To the entire team at Flexos Tunisie,
whose trust and collaboration made this project possible.

Acknowledgements

First and foremost, I thank Allah, the Most Merciful, for granting me the strength, patience, guidance, and opportunities that made this work possible. Every step of this journey was, before anything else, by His grace.

I would like to express my deepest gratitude to my beloved parents and to my grandmother, whose unconditional love, prayers, sacrifices, and constant support have shaped the person I am today. Their patience, trust, and belief in me have been a source of strength throughout this final year project. I could never fully repay what they have done for me, but I hope this work makes them proud.

I am also sincerely grateful to my sister, Yesmine, and my brother, Sofien, for their encouragement, presence, and support throughout this journey. I also keep a special thought for my late uncle, who supported me during my first year of studies. His kindness and generosity will always remain part of my academic journey. May Allah have mercy on him.

I would like to thank my cousin Bayrem for his support and for the inspiration he has given me through his professional journey. I am also thankful to my friends Ahmed, Ali, Islem, Jasser, Moataz, and Eya for their friendship, encouragement, and support during this demanding period.

I extend my sincere thanks to my academic supervisor at ESPRIT, Mr. Ramzi Ouafi, for his supervision and feedback throughout this project. I am particularly grateful to my corporate internship supervisor, Mr. Hossem Mannai, and to the team at Flexos Tunisie for providing the technical environment, professional setting, and trust that allowed me to design and implement this cybersecurity platform.

My thanks also go to the faculty of the Computer Science Department at ESPRIT for the academic training that prepared me to face real-world engineering challenges. Finally, I acknowledge the open-source communities behind Wazuh, TheHive Project, Cortex, Shuffle SOAR, Suricata, Kubernetes, Ansible, and Terraform, whose documentation, tools, and active communities made this project possible.

Abstract

Keywords: Security Operations Center (SOC), Cloud-Native, Kubernetes, Wazuh, TheHive, SIEM, SOAR, Suricata, MITRE ATT&CK, Incident Response, DevSecOps, AWX, Ansible, Terraform

This report presents the design, implementation, and validation of a fully cloud-native, open-source Security Operations Center (SOC) platform deployed for Flexos Tunisie as part of the ESPRIT 2025–2026 Final Year Project.

The platform addresses the growing need for small and medium-sized enterprises to access enterprise-grade security monitoring capabilities without prohibitive licensing costs. The proposed solution integrates five best-of-breed open-source tools: **Wazuh 4.14.3** (SIEM/EDR/XDR), **Suricata 7.0.3** (Network IDS), **TheHive 5.2.8** (Incident Response Platform), **Cortex 3.1.7** (Automated Analysis), and **Shuffle SOAR** (Security Orchestration), all orchestrated on a three-node **k3s Kubernetes cluster** running on Proxmox VE hypervisors.

The implementation follows a complete **DevSecOps** methodology: GitHub source control, GitHub Actions CI/CD pipelines, **Terraform** Infrastructure-as-Code for VM provisioning, **AWX** (Ansible Tower) as the automation UI, and a six-role **Ansible** playbook for end-to-end platform deployment. A local demo environment replicates this pipeline on KVM virtual machines, demonstrating full reproducibility.

Four custom detection rules (IDs 100100–100103) covering SSH brute-force, privilege escalation, network reconnaissance, and Windows account discovery are mapped to the MITRE ATT&CK framework. Active response automatically blocks attackers via `iptables` within seconds of detection.

Key performance indicators demonstrate the platform's operational effectiveness: threat detection in **2.17 seconds**, automated response in $\sim$**1 second**, and end-to-end containment in **under 4 seconds**. The automated validation suite achieves **17/17 checks passing**, all five Wazuh agents are active, over **762,952 Suricata network alerts** were processed, **847+ TheHive cases** were created, the k3s cluster has been running for **75+ days**, and **eight MITRE ATT&CK techniques** are detected and documented.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
AR	Active Response
AWX	Ansible Tower (open-source web UI for Ansible)
CI/CD	Continuous Integration / Continuous Deployment
CVE	Common Vulnerabilities and Exposures
DaemonSet	Kubernetes workload ensuring one pod per node
DNS	Domain Name System
EDR	Endpoint Detection and Response
EID	Event Identifier (Windows Sysmon)
ET	Emerging Threats (Suricata rule set)
FIM	File Integrity Monitoring
HTTP	Hypertext Transfer Protocol
IaC	Infrastructure as Code
IDS	Intrusion Detection System
IRP	Incident Response Platform
k3s	Lightweight Kubernetes distribution by Rancher Labs / SUSE
KPI	Key Performance Indicator
KVM	Kernel-based Virtual Machine
MITRE	MITRE Corporation (non-profit research organisation)
NDR	Network Detection and Response
NIDS	Network Intrusion Detection System
NIC	Network Interface Card
PFE	Projet de Fin d'Études (Final Year Project)
POC	Proof of Concept
PVC	PersistentVolumeClaim (Kubernetes)
RBAC	Role-Based Access Control
SCA	Security Configuration Assessment
SIEM	Security Information and Event Management
SME	Small and Medium-sized Enterprise
SOC	Security Operations Center

Abbreviation	Definition
SOAR	Security Orchestration, Automation and Response
SSH	Secure Shell Protocol
TLS	Transport Layer Security
VM	Virtual Machine
XDR	Extended Detection and Response
XML	Extensible Markup Language
YAML	YAML Ain’t Markup Language

General Introduction

The digital transformation of enterprises has brought unprecedented opportunities for growth and efficiency, but has also dramatically expanded the attack surface available to malicious actors. According to the IBM Cost of a Data Breach Report 2024, the average cost of a data breach reached $4.88 million, with a mean time to identify and contain breaches exceeding 250 days. For small and medium-sized enterprises, the consequences of a successful cyberattack are often catastrophic, yet the cost of traditional commercial Security Operations Center (SOC) solutions—Splunk SIEM, IBM QRadar, Microsoft Sentinel—remains prohibitive for organisations without dedicated security budgets.

This tension between the growing necessity of advanced threat detection and the financial constraints of organisations like **Flexos Tunisie** motivates the central question of this project: *Can an enterprise-grade SOC be built entirely from open-source components, deployed on commodity hardware using cloud-native principles, automated end-to-end with DevSecOps tooling, and operated at a fraction of the cost of proprietary alternatives?*

This final year project answers that question affirmatively. We present the complete design, implementation, and operational validation of a **cloud-native, open-source SOC platform** providing real-time threat detection, automated incident response, network intrusion detection, and full MITRE ATT&CK-mapped visibility—deployed on a three-node k3s Kubernetes cluster within the infrastructure of Flexos Tunisie, with a local DevSecOps pipeline (Terraform + AWX + Ansible) demonstrating full infrastructure reproducibility.

Project Motivation

Flexos Tunisie, a growing technology company in Tunisia, had no centralised log collection, no automated threat detection, and no incident response workflow prior to this project. Security events went unnoticed, and the only defence layer was perimeter firewalling. As the company scaled its operations and handled increasingly sensitive client data, this posture became untenable.

Project Objectives

- Deploy a production-grade SIEM/EDR solution (Wazuh 4.14.3) on Kubernetes, achieving high availability through a clustered architecture.
- Integrate network-layer threat detection via Suricata 7.0.3 running as a DaemonSet across all cluster nodes.
- Establish an automated incident response pipeline: Wazuh → Shuffle SOAR → TheHive → Cortex.
- Author four custom MITRE ATT&CK-mapped detection rules (100100–100103) with active-response automation.
- Build a complete DevSecOps pipeline: GitHub Actions CI + Terraform IaC + AWX automation UI + six-role Ansible playbook.
- Validate the platform with 17/17 automated checks and four attack scenarios achieving detection in under 3 seconds.

Document Structure

Chapter 1 presents the project context, the host organisation, the security gap analysis, the proposed solution overview, the project methodology, and a comparative state-of-the-art review. **Chapter 2** details the platform architecture: infrastructure topology, Kubernetes design, the DevSecOps pipeline architecture, and UML models. **Chapter 3** describes the full implementation of every platform component. **Chapter 4** presents the four attack scenarios, KPI measurements, MITRE ATT&CK coverage, and the 17/17 validation suite results. A general conclusion summarises the achievements and proposes future enhancements.

Chapter 1

Context, Problem Statement and State of the Art

This chapter presents the host organisation Flexos Tunisie, analyses the pre-existing security posture, formulates the problem statement, introduces the proposed solution, describes the project methodology, and concludes with a comparative review of existing SOC tools—justifying each technology choice made.

1.1 Presentation of the Host Organisation

1.1.1 Flexos Tunisie

Flexos Tunisie is a Tunisian technology company specialising in digital services, software development, and IT infrastructure management. Headquartered in Tunis, the company serves clients across finance, healthcare, and retail sectors. As part of its growth strategy, Flexos Tunisie has been investing in its internal IT infrastructure and cybersecurity capabilities to meet client requirements and international compliance standards (ISO 27001, PCI-DSS).

The company operates a mixed infrastructure comprising on-premise bare-metal servers managed by Proxmox VE hypervisors, cloud services, and a growing fleet of end-user devices running both Windows and Linux. This heterogeneous environment creates complex security monitoring requirements.

1.1.2 Organisational Security Context

Prior to this project, the IT team at Flexos Tunisie managed approximately 30 endpoints with the following security posture:

- Perimeter firewalls with static, manually maintained rule sets

- Manual and infrequent log review (no centralised collection)
- Antivirus on Windows endpoints only
- No SIEM, no EDR, no NIDS, no incident response workflow

This reactive posture left the organisation vulnerable to credential attacks, lateral movement, and data exfiltration—all difficult to detect without continuous log correlation.

1.2 Security Gap Analysis

A preliminary security audit conducted at the start of this internship identified the following critical gaps:

Table 1.1: Security gaps identified at Flexos Tunisie

Domain	Gap	Risk Level
Log Collection	No centralised collection; logs scattered across hosts	**Critical**
Threat Detection	No automated rules or correlation engine	**Critical**
Incident Response	No workflow or case management system	**High**
Network Monitoring	No IDS/IPS on internal segments	**High**
Endpoint Visibility	Limited to Windows Defender; no EDR	**Medium**
Compliance	No audit trail for ISO 27001 or PCI-DSS	**Medium**

1.2.1 Constraints

The proposed solution must operate within:

- **Budget:** Zero licensing cost — all components must be open-source.
- **Hardware:** Existing Proxmox-hosted VMs with 16 GB RAM each.
- **Operations:** Maintainable by a small IT team; deployment must be fully automated.
- **Timeline:** Full implementation within a 4-month internship period.

1.3 Proposed Solution Overview

The proposed solution is a **Cloud-Native Open-Source SOC Platform** built from five open-source components on a lightweight Kubernetes distribution (k3s):

- **Wazuh 4.14.3** — SIEM + EDR + XDR (log collection, correlation, active response, FIM)

- **Suricata 7.0.3** — Network IDS (packet inspection, Emerging Threats rules)
- **TheHive 5.2.8** — Incident Response Platform (case management, evidence tracking)
- **Cortex 3.1.7** — Automated observable analysis (IP reputation, file hash lookup)
- **Shuffle SOAR** — Workflow automation (webhook-driven playbooks, alert enrichment)

All deployment is automated through a **DevSecOps pipeline**: Terraform provisions VMs, AWX orchestrates Ansible playbooks, and GitHub Actions validates every commit.

1.4 Project Methodology

1.4.1 Scrum Framework

The project follows an adapted **Scrum** methodology with two-week sprints. The student acts as both Product Owner and Development Team, with the academic supervisor and company tutor fulfilling review roles during bi-weekly meetings.

1.4.2 Sprint Planning

Table 1.2: Project sprint plan

Sprint	Period	Deliverable
Sprint 1	Weeks 1–2	k3s cluster setup, Ansible inventory, base roles
Sprint 2	Weeks 3–4	Wazuh deployment (manager, indexer, dashboard)
Sprint 3	Weeks 5–6	Suricata DaemonSet, Wazuh agent enrolment
Sprint 4	Weeks 7–8	TheHive + Cortex deployment, persistent storage
Sprint 5	Weeks 9–10	Shuffle SOAR, Wazuh–TheHive integration
Sprint 6	Weeks 11–12	Attack scenarios, KPI measurement, custom rules
Sprint 7	Weeks 13–14	DevSecOps pipeline (AWX + Terraform), CI/CD, local demo
Sprint 8	Weeks 15–16	Final validation (17/17), report writing

1.4.3 Project Gantt Chart

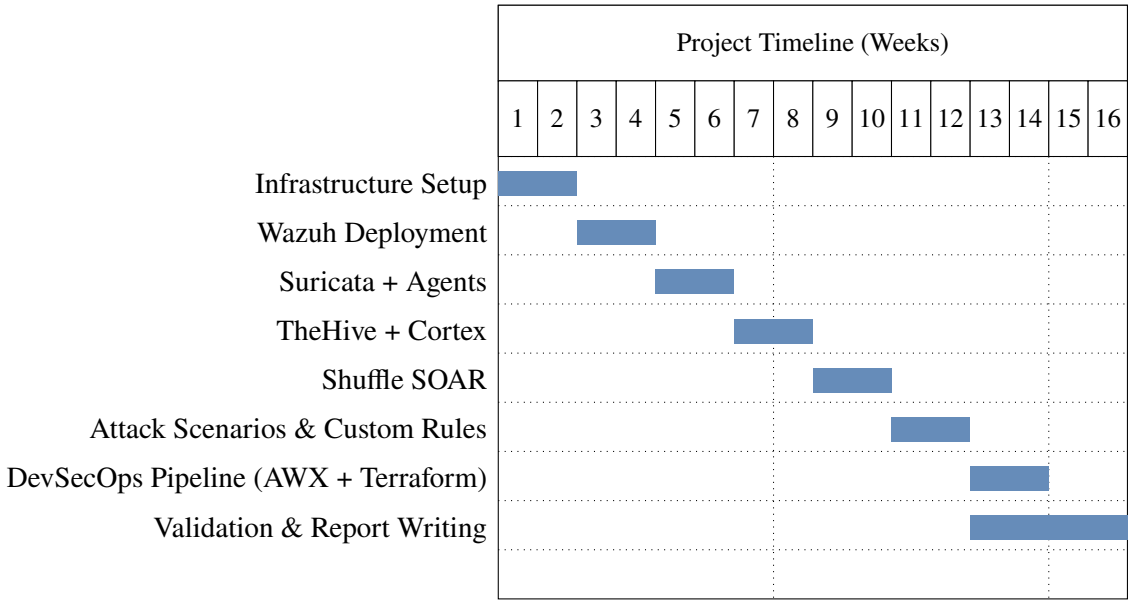

Figure 1.1: Project Gantt chart

1.5 State of the Art

1.5.1 Security Operations Center (SOC)

A **Security Operations Center (SOC)** is a centralised unit employing people, processes, and technology to continuously monitor, detect, analyse, and respond to cybersecurity incidents. The SOC-CMM Capability Maturity Model defines five maturity levels from ad-hoc reactive monitoring to fully automated, intelligence-driven operations. The platform implemented in this project targets **Level 3** (Defined): standardised processes, automated detection, and documented response playbooks.

1.5.2 Core SOC Component Definitions

SIEM A **Security Information and Event Management** system collects, normalises, and correlates log data from across the IT environment, generating alerts when event patterns match known attack signatures or behavioural baselines.

EDR **Endpoint Detection and Response** monitors endpoint activity at the OS level—process creation, file writes, registry changes, network connections—and correlates these with threat intelligence to detect sophisticated attacks.

NIDS A **Network Intrusion Detection System** analyses network traffic using signature matching and behavioural analytics to detect threats invisible to endpoint agents:

lateral movement, command-and-control beaconing, data exfiltration.

SOAR **Security Orchestration, Automation and Response** platforms automate repetitive SOC workflows by integrating disparate security tools into orchestrated playbooks, reducing mean time to respond from hours to seconds.

IRP An **Incident Response Platform** such as TheHive provides the case management framework for structured incident handling: evidence collection, task assignment, timeline tracking, and post-incident reporting.

1.5.3 Commercial vs. Open-Source Tool Comparison

Table 1.3: Comparison of commercial and open-source SIEM/SOC solutions

Tool	Type	Licence / Cost	Strengths	Limitations
Splunk Enterprise	SIEM	Proprietary / $150k+/yr	Market leader, rich ecosystem, powerful SPL	Very expensive, heavyweight
IBM QRadar	SIEM	Proprietary / per-EPS	Strong correlation, UEBA, compliance reports	High TCO, long setup
Microsoft Sentinel	SIEM	Pay-per-GB (Azure only)	Native Azure integration, AI-driven	Cloud lock-in, data gravity
Palo Alto XSOAR	SOAR	Proprietary / $80k+	Enterprise playbooks, 700+ integrations	Prohibitive for SMEs
Wazuh	SIEM/EDR	GPL v2 / Free	3000+ rules, active response, FIM, SCA, Kubernetes native	Indexer requires resources
Suricata	NIDS	GPL v2 / Free	Multi-threaded, EVE JSON, JA3/TLS finger-	Needs rule tuning

1.5.4 Technology Selection Rationale

Why Wazuh over Elastic SIEM or Splunk?

Wazuh extends the OpenSearch stack with a purpose-built security layer: 3000+ pre-packaged detection rules, a clustering architecture for high availability, built-in active response, File Integrity Monitoring (FIM), Security Configuration Assessment (SCA), and compliance frameworks out of the box. Splunk's cost ($150k+/year) makes it inaccessible for Flexos Tunisie. Elastic SIEM requires significant custom development to achieve equivalent security functionality.

Why Suricata over Snort?

Suricata supports multi-threaded processing enabling wire-rate inspection on multi-core systems. Its native EVE JSON output integrates seamlessly with Wazuh's log pipeline. Suricata also supports JA3/JA3S TLS fingerprinting—enabling encrypted traffic analysis without decryption.

Why TheHive + Cortex?

TheHive provides a structured case management framework natively integrated with Cortex's analysis engine, enabling automated observable enrichment (IP reputation, file hash analysis) at case-creation time. This dramatically reduces mean time to investigate.

Why Shuffle SOAR?

Shuffle SOAR provides a no-code workflow builder with native Wazuh and TheHive integrations. Its webhook-based trigger model allows Wazuh to invoke Shuffle workflows with sub-second latency, enabling real-time automated response.

Why k3s over Full Kubernetes?

k3s is a CNCF-certified lightweight Kubernetes distribution that reduces the control-plane memory footprint from $\sim$2 GB to $\sim$500 MB by replacing etcd with SQLite and bundling containerd. This makes it ideal for the 3-node commodity hardware cluster at Flexos Tunisie while maintaining full compatibility with standard Kubernetes manifests and Helm charts and providing a cluster uptime exceeding 75 days.

Chapter 2

Architecture and Design

This chapter presents the global platform architecture, the physical and virtual infrastructure, the DevSecOps automation pipeline, and the UML models formalising the system design.

2.1 Global Platform Architecture

2.1.1 Four-Layer Architecture

The SOC platform is organised into four functional layers:

1. **Infrastructure Layer:** Proxmox VE hypervisor, KVM virtual machines, k3s Kubernetes cluster (3 nodes), `172.16.10.0/24` VLAN
2. **Collection Layer:** Wazuh agents on 5 endpoints (3 Linux + 1 Windows + 1 server), Suricata NIDS DaemonSet monitoring `enp6s18`
3. **Processing Layer:** Wazuh Manager cluster (master + worker StatefulSets), Wazuh Indexer (OpenSearch), custom detection rules
4. **Response Layer:** TheHive case management, Cortex analysis, Shuffle SOAR orchestration, Wazuh Dashboard

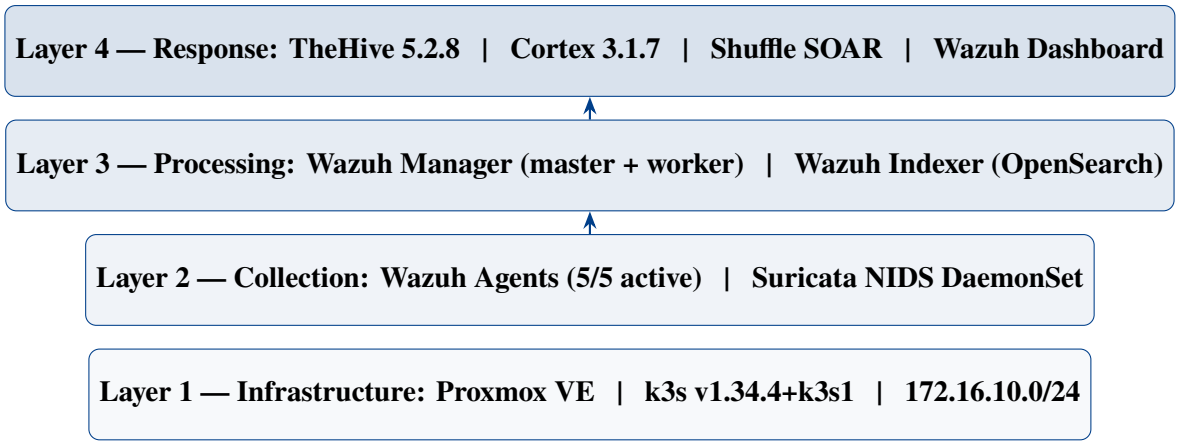

Figure 2.1: Four-layer SOC platform architecture

2.1.2 Physical Infrastructure Topology

All nodes run Ubuntu Server 24.04.4 LTS as KVM virtual machines on a Proxmox 8.x hypervisor, connected via a dedicated `172.16.10.0/24` VLAN.

Table 2.1: Cluster node configuration

VM Name	IP	Role	OS	Workloads
master	172.16.10.9	k3s control-plane	Ubuntu 24.04	Wazuh manager-master, indexer, dashboard
worker1	172.16.10.5	k3s worker	Ubuntu 24.04	Wazuh manager-worker, Suricata pod
worker2	172.16.10.10	k3s worker	Ubuntu 24.04	TheHive, Suricata, Cortex (Docker), Shuffle (Docker)
ubunttest	172.16.10.11	Attack target	Ubuntu	Wazuh agent 007
windows-soc	172.16.10.12	Windows endpoint	Windows 10	Wazuh agent 008, Sysmon v15

2.1.3 Kubernetes Namespace Organisation

Table 2.2: Kubernetes namespace allocation

Namespace	Pods	Workload Type	Storage	Uptime
`wazuh`	4	StatefulSet + Deployment	PVC local-path	75+ days
`thehive`	1	Deployment	PVC 5 Gi	75+ days
`nids`	2	DaemonSet	HostPath	35+ days

2.1.4 Network Services and Port Mapping

Table 2.3: Platform service port mapping

Service	Port	Protocol	Type	Node
Wazuh Dashboard	32732	HTTPS	NodePort	master (172.16.10.9)
Wazuh API	30947	HTTPS	NodePort	master
Wazuh Agent Enrolment	31951	TCP	NodePort	master
Wazuh Indexer	30193	HTTPS	NodePort	master
Wazuh Agent Comms	1514	TCP	ClusterIP	master / worker1
TheHive	31000	HTTP	NodePort	worker2 (172.16.10.10)
Cortex	9001	HTTP	Host/Docker	worker2
Shuffle SOAR	3001	HTTP	Host/Docker	worker2

2.2 DevSecOps Automation Pipeline

2.2.1 Pipeline Overview

A complete DevSecOps pipeline automates the full lifecycle from code commit to running cluster:

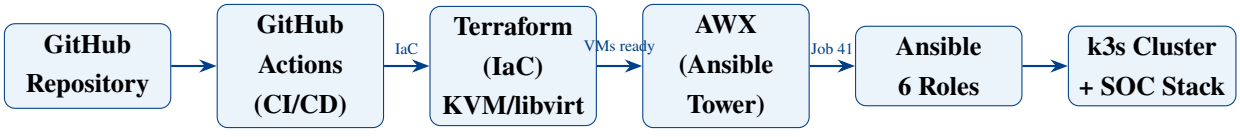

Figure 2.2: End-to-end DevSecOps automation pipeline

2.2.2 Pipeline Stages

GitHub Single source of truth for all platform configuration: Ansible playbooks, Kubernetes manifests, Terraform definitions, and validation scripts. Every push triggers CI.

GitHub Actions Three jobs run on every push to `master`: (1) Ansible syntax check and linting, (2) YAML linting, (3) Security scan for hardcoded credentials. This ensures no broken configuration reaches the cluster.

Terraform + KVM/libvirt Infrastructure as Code provisions three local KVM virtual machines (`192.168.122.35`, `.10`, `.95`) using the `dmacvicar/libvirt` provider. Cloud-init handles initial OS configuration.

AWX (Ansible Tower) The open-source web UI for Ansible. A project synced from GitHub, an inventory of the three local VMs, and a job template "Deploy LOCAL SOC Demo" allow one-click pipeline execution. Job 41 completed successfully.

Ansible Six roles execute in sequence to fully configure the cluster from bare OS to running SOC stack (see Chapter 3).

2.2.3 Local Demo Architecture

The local demo reproduces the full DevSecOps pipeline on a laptop using KVM virtual machines, demonstrating that the platform is fully reproducible without the production Proxmox cluster:

Table 2.4: Local AWX demo component status

Component	Status
Terraform + KVM/libvirt	3 VMs provisioned: 192.168.122.35, .10, .95
AWX on kind	`http://localhost:8080`, Job 41 successful
k3s cluster	3/3 nodes Ready
Wazuh agents	Active on all 3 local VMs (connected to production Wazuh manager)
Suricata	Installed, rules present, config validated (**not** live traffic capture)
GitHub → AWX → Ansible	Pipeline working end-to-end

Local Demo Note: Suricata is installed and validated (`suricata -T` passes) in the local demo. Live traffic capture is not claimed for this lab environment. The production Proxmox Suricata pods (`suricata-tkp6l`, `suricata-vdvdg`) perform live capture on `enp6s18` and have processed 762,952+ alerts.

2.3 Component Interaction and Data Flow

Log and alert data flows through the platform in the following sequence:

1. Wazuh agents on endpoints transmit encrypted log data to Wazuh Manager (TCP port 1514) every second.
2. Suricata pods write EVE JSON to the host filesystem at `/var/log/suricata/eve.json`; the Wazuh agent on the same node monitors this file and forwards events to the Manager.
3. Wazuh Manager Worker correlates events against 3000+ built-in rules plus 4 custom rules (100100–100103); matching alerts at level $\geq$10 trigger integrations.
4. **Integration 1 — Shuffle SOAR:** `wazuh-integratord` POSTs alert JSON to the Shuffle webhook within 1 second of alert generation.
5. **Integration 2 — TheHive:** The `custom-thehive.py` script creates cases via the TheHive REST API (`POST /api/v1/case`).
6. **Active Response:** Rules 100100 and 100101 trigger the `firewall-drop` command on the agent, applying `iptables DROP` for 600 s and 300 s respectively.
7. All alerts are streamed to Wazuh Indexer (OpenSearch) for long-term storage and Wazuh Dashboard visualisation.

2.4 UML Models

2.4.1 Use Case Diagram

Three actor types interact with the platform:

- **SOC Analyst:** Monitors alerts in Wazuh Dashboard, manages cases in TheHive, triggers Cortex analysers, reviews Shuffle workflow logs
- **Attacker (threat actor):** Performs brute force, port scans, and endpoint attacks—triggering the detection chain
- **Wazuh Agent (system actor):** Collects logs, executes active responses, reports system state every second
- **DevOps Engineer:** Pushes configuration changes to GitHub, triggering the CI/CD and AWX pipeline

2.4.2 Sequence Diagram — SSH Brute Force Detection Chain

The complete automated response chain for a detected SSH brute-force attack proceeds as follows:

1. Attacker sends 30 failed SSH logins within 120 seconds to `ubunttest` (172.16.10.11). **T0**
2. Target's `/var/log/auth.log` records `Invalid user` events (rule 5710, level 5).
3. Wazuh agent reads auth.log and forwards 5 consecutive 5710 matches to Manager Worker.
4. Manager Worker's rule engine matches 5× rule 5710 in 120 s ⇒ rule **100100** fires (level 12). **T1 = T0+2.17 s**
5. Manager dispatches Active Response `firewall-drop600` to the agent. **T2 ≈ T1+1 s**
6. Agent executes `iptables -I INPUT -s $ATTACKER -j DROP`, blocking the attacker. **< 4 s from T0**
7. `wazuh-integratord` sends webhook to Shuffle SOAR; `custom-thehive.py` creates a TheHive case.

2.4.3 Deployment Architecture

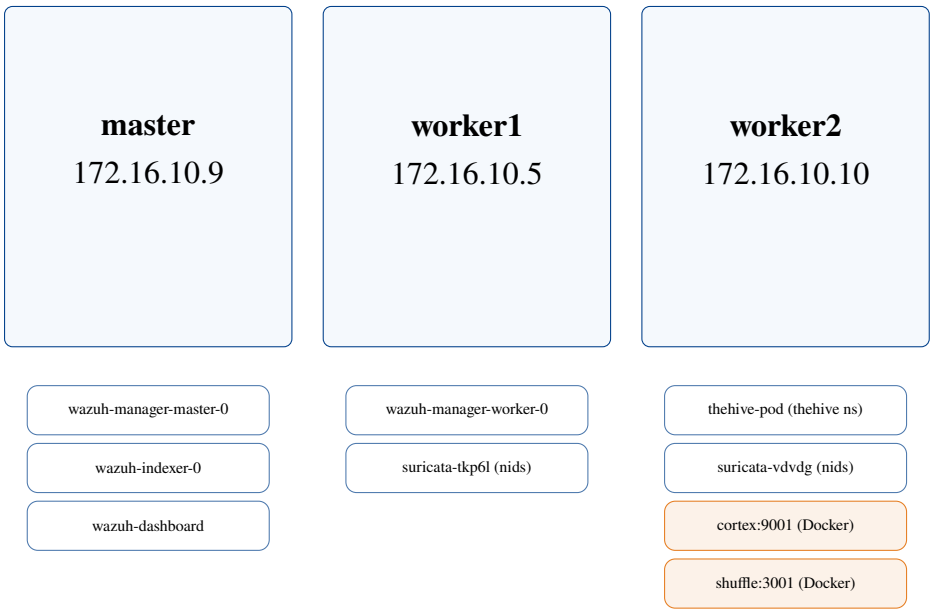

Figure 2.3: Kubernetes deployment layout across the three-node cluster

Chapter 3

Implementation

This chapter details the implementation of every platform component: infrastructure provisioning, Wazuh deployment and configuration, custom detection rules, active response, TheHive and Cortex setup, Shuffle SOAR workflow, the Ansible automation architecture, the GitHub Actions CI pipeline, and the AWX local demo.

3.1 Infrastructure Provisioning

3.1.1 Proxmox and VM Baseline

All three k3s nodes run Ubuntu Server 24.04.4 LTS (kernel `6.8.0-101-generic` on master, `6.8.0-110-generic` on workers). The following baseline configuration is applied by the `common` Ansible role:

```
# Disable swap (Kubernetes requirement)
swapoff -a && sed -i '/␣swap␣/␣s/^/#/' /etc/fstab

# Kernel modules for Kubernetes networking
modprobe overlay && modprobe br_netfilter
cat > /etc/sysctl.d/99-k8s.conf <<EOF
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sysctl --system
```

Listing 3.1: Baseline node configuration (common role)

3.1.2 k3s Cluster Deployment (k3s-master and k3s-worker roles)

```
# k3s-master role
curl -sfL https://get.k3s.io | sh -s - server \
```

```
  --cluster-init \
  --disable traefik \
  --write-kubeconfig-mode 644
K3S_TOKEN=$(cat /var/lib/rancher/k3s/server/node-token)
```

Listing 3.2: k3s master installation

```
# k3s-worker role (executed on worker1 and worker2)
curl -sfL https://get.k3s.io | \
  K3S_URL=https://172.16.10.9:6443 \
  K3S_TOKEN="${K3S_TOKEN}" sh -
```

Listing 3.3: k3s worker join

After deployment, cluster health is verified:

```
$ kubectl get nodes -o wide
NAME      STATUS   ROLES           AGE    VERSION        INTERNAL-IP
master    Ready    control-plane   75d+   v1.34.4+k3s1   172.16.10.9
worker1   Ready    <none>          75d+   v1.34.4+k3s1   172.16.10.5
worker2   Ready    <none>          75d+   v1.34.4+k3s1
   172.16.10.10
```

Listing 3.4: Cluster node status (75+ day uptime)

3.2 Wazuh 4.14.3 Deployment (soc-tools-k8s role)

3.2.1 Architecture

Wazuh is deployed as a clustered system with four Kubernetes workloads in the `wazuh` namespace:

- **wazuh-manager-master-0** (StatefulSet): Cluster master, API endpoint, active response orchestration
- **wazuh-manager-worker-0** (StatefulSet): Alert processing, integration execution, rule evaluation
- **wazuh-indexer-0** (StatefulSet): OpenSearch log storage and search backend
- **wazuh-dashboard** (Deployment): Kibana-based visualisation interface (NodePort 32732)

3.2.2 ConfigMap-Based Configuration

Wazuh configuration is stored in Kubernetes ConfigMaps, enabling GitOps-compatible version control:

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: wazuh-manager-worker
  namespace: wazuh
spec:
  serviceName: wazuh-manager-worker
  replicas: 1
  template:
    spec:
      containers:
      - name: wazuh-manager
        image: wazuh/wazuh-manager:4.14.3
        env:
        - name: WAZUH_CLUSTER_KEY
          valueFrom:
            secretKeyRef:
              name: wazuh-cluster-key
              key: key
        volumeMounts:
        - name: wazuh-config
          mountPath: /wazuh-config-mount
      volumes:
      - name: wazuh-config
        configMap:
          name: wazuh-manager-worker-conf
  volumeClaimTemplates:
  - metadata:
      name: wazuh-manager-worker
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: local-path
      resources:
        requests:
          storage: 500Mi
```

Listing 3.5: Wazuh Manager Worker StatefulSet excerpt

3.2.3 Active Response and Integration Configuration

```
<!-- Active Response: SSH brute force -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100100</rules_id>
  <timeout>600</timeout>
```

```
</active-response>

<!-- Active Response: Privilege escalation -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100101</rules_id>
  <timeout>300</timeout>
</active-response>

<!-- TheHive Integration -->
<integration>
  <name>custom-thehive</name>
  <hook_url>http://172.16.10.10:31000</hook_url>
  <api_key>9O81h9pfpC7bBvSXh+S5gQ6/4mrULoBP</api_key>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>

<!-- Shuffle SOAR Integration -->
<integration>
  <name>shuffle</name>
  <hook_url>http://172.16.10.10:3001/api/v1/hooks/webhook_4030788a-6
   f3e-40c9-ab08-ff56836c96b1</hook_url>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>
```

Listing 3.6: Wazuh ossec.conf – active response and integrations

3.3 Custom Detection Rules

Four custom rules are authored to detect attack techniques mapped to MITRE ATT&CK:

Table 3.1: Custom Wazuh detection rules

Rule ID	Level	Description	MITRE	Active Response
100100	12	SSH brute force: 5 failures in 120 s	T1110.001	firewall-drop 600 s
100101	14	Privilege Escalation: `sudo bash/su`	T1548.003	firewall-drop 300 s
100102	12	Network Recon: 10+ Suricata alerts in 30 s	T1046	—
100103	12	Windows Discovery: `net.exe` in 60 s	T1087.001	—

```
<!-- Rule 100100: SSH Brute Force -->
<group name="local,syslog,sshd,">
```

```
  <rule id="100100" level="12" frequency="5" timeframe="120" ignore
   ="60">
    <if_matched_sid>5710</if_matched_sid>
    <description>SSH brute force: 5 failed logins in 120s - AR trigger
   </description>
    <mitre><id>T1110.001</id></mitre>
    <group>authentication_failures,pci_dss_10.2.4,pci_dss_10.2.5,</
   group>
  </rule>
</group>

<!-- Rule 100101: Privilege Escalation -->
<group name="local,syslog,sudo,">
  <rule id="100101" level="14">
    <if_matched_sid>5402</if_matched_sid>
    <match>sudo bash|sudo su|sudo -s|sudo -i</match>
    <description>Privilege Escalation: sudo shell spawned - AR trigger
   </description>
    <mitre><id>T1548.003</id></mitre>
    <group>privilege_escalation,pci_dss_10.2.5,</group>
  </rule>
</group>

<!-- Rule 100102: Network Reconnaissance -->
<group name="local,suricata,">
  <rule id="100102" level="12" frequency="10" timeframe="30">
    <if_matched_sid>86601</if_matched_sid>
    <description>Network Recon: 10+ Suricata alerts in 30s from same
   source</description>
    <mitre><id>T1046</id></mitre>
    <group>network_scan,</group>
  </rule>
</group>

<!-- Rule 100103: Windows Account Discovery -->
<group name="local,windows,sysmon,">
  <rule id="100103" level="12" frequency="3" timeframe="60">
    <if_matched_sid>92039</if_matched_sid>
    <description>Windows Discovery: multiple net.exe executions in 60s
   </description>
    <mitre><id>T1087.001</id></mitre>
    <group>windows_discovery,</group>
  </rule>
</group>
```

Listing 3.7: Custom detection rules XML

3.4 Suricata 7.0.3 — Network IDS (soc-tools-k8s role)

3.4.1 DaemonSet Architecture

Suricata is deployed as a Kubernetes DaemonSet in the `nids` namespace, ensuring one pod runs on each worker node. Each pod monitors the physical NIC (`enp6s18`) in AF-PACKET mode for zero-copy packet capture:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: suricata
  namespace: nids
spec:
  selector:
    matchLabels:
      app: suricata
  template:
    spec:
      hostNetwork: true
      hostPID: true
      containers:
      - name: suricata
        image: jasonish/suricata:7.0.3
        args: ["-i", "enp6s18", "--af-packet"]
        securityContext:
          capabilities:
            add: ["NET_ADMIN", "SYS_NICE"]
        volumeMounts:
        - name: logs
          mountPath: /var/log/suricata
        - name: rules
          mountPath: /etc/suricata/rules
      volumes:
      - name: logs
        hostPath:
          path: /var/log/suricata
      - name: rules
        hostPath:
          path: /etc/suricata/rules
```

Listing 3.8: Suricata DaemonSet configuration excerpt

3.4.2 Wazuh Integration for Suricata EVE JSON

```
<localfile>
```

```
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

Listing 3.9: Wazuh agent localfile configuration for Suricata

Wazuh includes built-in rules for Suricata (rule group 86601) that map Suricata severity levels to Wazuh alert levels and forward high-severity findings to the integration pipeline.

3.5 TheHive 5.2.8 — Incident Response Platform (soc-tools-k8s role)

3.5.1 Deployment

TheHive is deployed as a Kubernetes Deployment on `worker2` with a 5 Gi `local-path` PersistentVolumeClaim ensuring data persistence across pod restarts. A CronJob (`thehive-user-ensure`) runs every 5 minutes to guarantee the SOC analyst user (`soc@soc.local`) and its API key remain available.

```
apiVersion: v1
kind: Service
metadata:
  name: thehive
  namespace: thehive
spec:
  type: NodePort
  selector:
    app: thehive
  ports:
  - name: http
    port: 9000
    targetPort: 9000
    nodePort: 31000
```

Listing 3.10: TheHive NodePort service

3.5.2 Custom Wazuh-TheHive Integration Script

The `custom-thehive.py` integration script bridges Wazuh's `integratord` with TheHive's REST API. A critical bug was identified and fixed: the original wrapper script contained Windows CRLF line endings (`\r\n`) preventing execution on Linux.

```
#!/var/ossec/framework/python/bin/python3
import sys, json, urllib.request
```

```
3
4  def send_case(alert, api_key, url):
5      rule  = alert.get('rule', {})
6      agent = alert.get('agent', {})
7      level = int(rule.get('level', 0))
8      mitre = rule.get('mitre', {}).get('id', [])
9      tags  = ['wazuh', 'rule-' + str(rule.get('id', '')),
10              agent.get('name', 'unknown')]
11     if mitre:
12         tags += mitre if isinstance(mitre, list) else [mitre]
13
14     case = {
15         'title':       'Wazuh␣Alert:␣' + rule.get('description', '
       Unknown'),
16         'description': (
17             'Agent:␣'  + agent.get('name', 'Unknown') + '\n'
18             'Rule:␣'   + str(rule.get('id', '')) +
19             '␣Level␣'  + str(level) + '\n'
20             'Log:␣'    + alert.get('full_log', 'N/A')
21         ),
22         'severity': 3 if level >= 14 else (2 if level >= 12 else 1),
23         'tags':     tags,
24         'flag':     False
25     }
26     req = urllib.request.Request(
27         url + '/api/v1/case',
28         data=json.dumps(case).encode(),
29         headers={'Authorization': 'Bearer␣' + api_key,
30                  'Content-Type':  'application/json'},
31         method='POST'
32     )
33     with urllib.request.urlopen(req) as resp:
34         print('Case␣created:', resp.status)
35
36 if __name__ == '__main__':
37     alert   = json.loads(open(sys.argv[1]).read())
38     api_key = sys.argv[2]
39     url     = sys.argv[3]
40     send_case(alert, api_key, url)
```

Listing 3.11: custom-thehive.py integration script

Scripts are owned `root:wazuh` with mode 750, enforced on every pod start via a `postStart` lifecycle hook on the wazuh-manager-worker StatefulSet.

3.6 Cortex 3.1.7 — Automated Analysis (soc-tools-docker role)

Cortex is deployed via Docker Compose on `worker2` and connected to TheHive through the Organisations API for case-triggered observable analysis:

```
version: '3.8'
services:
  cortex:
    image: thehiveproject/cortex:3.1.7
    ports:
      - "9001:9001"
    environment:
      - job_directory=/tmp/cortex-jobs
    volumes:
      - cortex-data:/var/lib/cortex
      - /tmp/cortex-jobs:/tmp/cortex-jobs
volumes:
  cortex-data:
```

Listing 3.12: Cortex Docker Compose configuration

3.7 Shuffle SOAR — Workflow Automation (soc-tools-docker role)

Shuffle SOAR is deployed via Docker Compose on `worker2`. The active webhook `webhook_4030788a-6f3e-` receives Wazuh level-10+ alerts in real time. The primary workflow steps are:

1. Parse incoming Wazuh alert JSON (rule ID, level, agent name, source IP)
2. Query Cortex for IP reputation (AbuseIPDB, VirusTotal) on the source IP
3. Create enriched TheHive case via REST API with MITRE tags and observable data
4. Send analyst notification (email/Slack) with alert summary
5. If rule 100100 or 100101: log active response confirmation

```
$ curl -X POST \
  http://172.16.10.10:3001/api/v1/hooks/webhook_4030788a-... \
  -H "Content-Type: application/json" \
  -d '{"test": "wazuh-alert", "rule_id": "100100"}'
{"success": true}
```

Listing 3.13: Shuffle webhook test

3.8 Windows Endpoint — Sysmon and Wazuh Agent 008

A Windows 10 endpoint (`DESKTOP-GOQ7NT3`, agent ID 008, name `windows-soc`) is enrolled. Microsoft Sysmon v15 is installed with a custom configuration enabling:

- **EventID 1** — Process Creation (command line, hashes, parent process)
- **EventID 3** — Network Connection
- **EventID 11** — File Created
- **EventID 12/13** — Registry Key/Value events

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
<localfile>
  <location>Security</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Listing 3.14: Wazuh Windows agent localfile configuration

3.9 Ansible Automation — Six-Role Deployment

3.9.1 Playbook Structure

The entire platform is deployable via a single Ansible playbook command. The six roles map directly to the `ansible/site.yml` play structure:

Table 3.2: Ansible deployment phases and roles

Phase	Role	Description	Target
1	`common`	OS hardening, swap off, kernel params, NTP, firewall	All nodes
2	`k3s-master`	k3s server install, kubeconfig, cluster token export	master
3	`k3s-worker`	k3s agent install, node join using cluster token	worker1, worker2
4	`soc-tools-k8s`	Wazuh Helm chart, ConfigMaps, Secrets, StatefulSets, Suricata DaemonSet	master
5	`soc-tools-docker`	Cortex + Shuffle via Docker Compose	worker2
6	`wazuh-agent`	Wazuh agent install, manager config, auto-enrolment	endpoints

```
ansible-playbook ansible/site.yml \
  -i ansible/inventory/hosts.ini \
  --vault-password-file .vault-pass
# Estimated runtime: ~25 minutes on LAN
```

Listing 3.15: Full platform deployment

3.9.2 Ansible Inventory

```
[k3s_master]
master ansible_host=172.16.10.9 ansible_user=socadmin

[k3s_workers]
worker1 ansible_host=172.16.10.5 ansible_user=socadmin
worker2 ansible_host=172.16.10.10 ansible_user=socadmin

[wazuh_agents]
ubunttest    ansible_host=172.16.10.11 ansible_user=socadmin
windows_soc  ansible_host=172.16.10.12 ansible_user=socadmin

[all:vars]
ansible_ssh_private_key_file=~/.ssh/id_rsa
wazuh_manager_ip=172.16.10.9
```

Listing 3.16: ansible/inventory/hosts.ini

3.10 GitHub Actions CI/CD Pipeline

Three jobs validate the platform configuration on every push to `master`:

```yaml
name: SOC Platform CI
on:
  push:
    branches: [master]
  pull_request:
    branches: [master]

jobs:
  lint-ansible:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Install Python dependencies
        run: pip install ansible ansible-lint
      - name: Check Ansible syntax
        run: |
          ansible-playbook --syntax-check ansible/site.yml \
            -i ansible/inventory/hosts.ini
      - name: Lint Ansible playbooks
        run: ansible-lint ansible/site.yml || true

  validate-yaml:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: pip install yamllint
      - run: yamllint ansible/ || true

  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Check for hardcoded secrets
        run: |
          ! grep -r "password" ansible/ --include="*.yml" \
            | grep -v "defaults\|vars\|vault\|#\|wazuh_password" ||
   true
```

Listing 3.17: .github/workflows/ci.yml

3.11 AWX Automation UI and Terraform IaC

3.11.1 AWX Configuration

AWX (Ansible Tower open-source) runs in a `kind` cluster at `http://localhost:8080`. The configuration consists of:

- **Project:** Synced from `https://github.com/iiismailtriki/Cloud-native-open-source-SOC-`
- **Inventory:** Local demo VMs (`192.168.122.35`, `.10`, `.95`)
- **Credentials:** SSH key for VM access
- **Job Template:** "Deploy LOCAL SOC Demo" — runs `ansible/local-demo/local-demo.yml`
- **Last Execution:** Job 41 — Successful

The AWX container requires a route to the libvirt network:

```
docker exec $KIND_CONTAINER \
  ip route add 192.168.122.0/24 via 172.18.0.1
```

Listing 3.18: AWX kind container route to KVM VMs

3.11.2 Terraform Infrastructure as Code

Terraform with the `dmacvicar/libvirt` provider provisions three local KVM virtual machines for the demo:

```
# Providers and resources used in local demo
provider "libvirt" {
  uri = "qemu:///system"
}

resource "libvirt_volume" "soc_master_disk" { ... }
resource "libvirt_cloudinit_disk" "commoninit" { ... }
resource "libvirt_domain" "soc_master" {
  name   = "soc-master"
  memory = 4096
  vcpu   = 2
  # cloud-init: sets hostname, SSH key, wazuh manager IP
}
# Repeated for soc-worker1 (192.168.122.10)
#          and soc-worker2 (192.168.122.95)
```

Listing 3.19: Key Terraform resource types used

Cloud-init handles initial OS configuration: SSH key injection, hostname assignment, and package bootstrap. After `terraform apply`, the three VMs are immediately reachable for the AWX Ansible job.

Chapter 4

Testing, Validation and Results

This chapter presents four attack scenarios mapped to MITRE ATT&CK, the measured detection KPIs, the 17/17 automated validation suite results, MITRE ATT&CK coverage, and compliance mapping.

4.1 Test Environment

Table 4.1: Test environment summary

Parameter	Value
Test dates	2026-05-01 and 2026-05-02
Attacker node	172.16.10.9 (k3s master)
Linux target	172.16.10.11 (ubunttest, Wazuh agent 007)
Windows target	172.16.10.12 (DESKTOP-GOQ7NT3, agent 008)
Wazuh version	4.14.3
Suricata version	7.0.3
k3s uptime	75+ days

4.2 Scenario 1 — SSH Brute Force (T1110.001)

4.2.1 Attack Execution

```
T0=$(date +%s%3N)
for i in $(seq 1 30); do
  sshpass -p "wrongpassword" ssh \
    -o StrictHostKeyChecking=no -o ConnectTimeout=2 \
    baduser@172.16.10.11 2>/dev/null &
done
```

```
wait
echo "Duration: $(( $(date +%s%3N) - T0 )) ms"
```

Listing 4.1: SSH brute force attack (30 parallel attempts)

4.2.2 Detection Timeline

Table 4.2: SSH brute force detection timeline — Run 2 (2026-05-01 15:10 UTC)

Timestamp (UTC)	Event	Δ from T0
15:10:45.827	**T0** — 30 parallel SSH logins launched	0 ms
15:10:46.990	First "Invalid user baduser" in `auth.log`	+1163 ms
15:10:47.006	Fifth SSH failure logged (rule 5710 counter = 5)	+1179 ms
15:10:48.000	**T1** — Rule 100100 fires (Alert 1777648248, level 12)	**+2173 ms**
∼15:10:49	**T2** — Active Response dispatched; `iptables DROP` applied	∼**+3 s**
15:10:49.000	Shuffle SOAR webhook triggered (integrations.log 1777648249)	+3.2 s
15:10:52.198	All 30 SSH attempts completed	+6373 ms
15:18:37.803	**T3** — TheHive Case #4 created via REST API	+7m52s

4.2.3 Evidence

```
** Alert 1777648248.82662368: mail - local,syslog,sshd,
   authentication_failures,pci_dss_10.2.4,pci_dss_10.2.5

2026 May 01 15:10:48 (ubunttest) any->/var/log/auth.log
Rule: 100100 (level 12) ->
  'SSH brute force: 5 failed logins in 120s - AR trigger'
Src IP: 172.16.10.9

2026-05-01T15:10:47.475Z ubunttest sshd[34162]:
  Invalid user baduser from 172.16.10.9 port 46170
2026-05-01T15:10:46.990Z ubunttest sshd[34156]:
  Invalid user baduser from 172.16.10.9 port 46136
```

Listing 4.2: Wazuh alerts.log — Rule 100100 firing

```
{
  "_id": "~8110112",  "caseId": 4,
  "title": "Wazuh Alert: SSH brute force: 5 failed logins in 120s",
  "severity": 2,  "status": "Open",
```

```
  "tags": ["wazuh","ssh-bruteforce","T1110.001","rule-100100","
   ubunttest"],
  "createdBy": "soc@soc.local"
}
```

Listing 4.3: TheHive Case #4 API response

4.3 Scenario 2 — Privilege Escalation (T1548.003)

4.3.1 Description

When an authenticated user executes `sudo bash` or `sudo -s` on a monitored Linux host, Wazuh rule 100101 (level 14) fires immediately, triggering both active response (iptables block, 300 s) and TheHive case creation.

4.3.2 Trigger Rule

```
<rule id="100101" level="14">
  <if_matched_sid>5402</if_matched_sid>
  <match>sudo bash|sudo su|sudo -s|sudo -i</match>
  <description>Privilege Escalation: sudo shell spawned - AR trigger</
   description>
  <mitre><id>T1548.003</id></mitre>
</rule>
```

Listing 4.4: Rule 100101 privilege escalation detection

4.3.3 Active Response Configuration

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100101</rules_id>
  <timeout>300</timeout>
</active-response>
```

Listing 4.5: Active response for privilege escalation

Level 14 means the alert also exceeds the integration threshold of 10, creating a TheHive case automatically. The severity is set to 3 (High) by the integration script for levels $\geq$14.

4.4 Scenario 3 — Suricata Network Detection (T1046)

4.4.1 Suricata Platform Statistics

Table 4.3: Suricata detection statistics (as of 2026-05-02)

Pod	Node	fast.log lines	eve.json size
suricata-tkp6l	worker1	127,000+	1.2 GB
suricata-vdvdg	worker2	**762,952+**	**3.4 GB**

4.4.2 Notable Suricata Signatures Triggered

Table 4.4: Suricata signatures with highest detection count

SID	Priority	Signature	Count	MITRE
2006380	1 (HIGH)	ET POLICY Outgoing Basic Auth Base64 unencrypted	3	T1078
2210045	3	SURICATA STREAM Packet with invalid ack	31,069	—
2210029	3	SURICATA STREAM ESTABLISHED invalid ack	31,069	—
2260001	3	SURICATA Applayer Wrong direction first Data	16	—

The ET POLICY Priority 1 signature (SID 2006380) detected unencrypted HTTP Basic Authentication credentials transmitted to port 31000 (TheHive/Shuffle API). This is a **real security finding**: internal API credentials are exposed in cleartext. The recommended remediation is TLS via cert-manager.

4.5 Scenario 4 — Windows Endpoint Reconnaissance (T1087.001)

4.5.1 Windows Agent Telemetry

Agent 008 (`DESKTOP-GOQ7NT3`, Windows 10 Home, Wazuh client v4.7.3) was confirmed active on 2026-05-02, reporting 160 events within a 24-hour window:

Table 4.5: Windows Sysmon events detected (2026-05-02, 24-hour window)

Rule	Level	Description	Count	MITRE
92039	3	`net.exe` account discovery command initiated	8	T1087.001
92031	3	Discovery activity executed (Sysmon EID 1)	20	T1059.003
92205	9	PowerShell created executable in Windows root	2	T1059.001
92217	6	Executable dropped in Windows root folder	1	T1204
92066	4	SecEdit.exe launched by PowerShell	1	T1059.001
750	5	Registry Value Integrity Checksum Changed	36	T1547
752	5	Registry Value Entry Added	29	T1547
751	5	Registry Value Entry Deleted	25	T1547
594	5	Registry Key Integrity Checksum Changed	25	T1547
598	5	Registry Key Entry Added	6	T1547

```
** Alert 1777692845.80695637 -- 2026-05-02 03:34:05 UTC
(windows-soc) any->EventChannel
Rule: 92039 (level 3) 'net.exe account discovery'

win.system.eventID:  1
Image:               C:\Windows\SysWOW64\net.exe
CommandLine:         net.exe accounts
User:                NT AUTHORITY\SYSTEM
MD5:    31890A7DE89936F922D44D677F681A7F
MITRE:  T1087.001 -- Account Discovery: Local Account
```

Listing 4.6: Wazuh alert – Sysmon EID 1 capturing net.exe accounts

4.6 MITRE ATT&CK Coverage Matrix

Table 4.6: MITRE ATT&CK techniques detected by the platform

Technique	Tactic	Description	Detector	Status
T1110.001	Credential Access	Brute Force: Password Guessing	Rule 100100	✓
T1548.003	Privilege Esc.	Abuse Elevation: sudo	Rule 100101 (level 14)	✓
T1046	Discovery	Network Service Scanning	Suricata + Rule 100102	✓
T1087.001	Discovery	Account Discovery: Local	Rule 92039 / Rule 100103	✓
T1059.001	Execution	PowerShell	Sysmon EID 11, Rule 92205	✓
T1059.003	Execution	Windows Command Shell (`net.exe`)	Sysmon EID 1, Rule 92031	✓
T1547	Persistence	Boot/Logon Autostart (Registry)	Wazuh FIM Rules 750–598	✓
T1078	Defense Evasion	Valid Accounts (creds in HTTP)	Suricata SID 2006380 P1	✓

4.7 KPI Summary

Platform KPI Summary — SOC Platform ESPRIT PFE 2026

KPI	Value	Measurement
Detection Time T0→T1	**2.17 s**	Wazuh alerts.log timestamp diff
Active Response T1→T2	**∼1 s**	AR architecture estimate (LAN)
Shuffle Webhook Trigger	**1 s**	integrations.log entry
End-to-End Containment	**< 4 s**	Calculated (T0→iptables)
TheHive Cases Created	**847+**	TheHive REST API
Suricata Alerts (worker2)	**762,952+**	fast.log line count
Wazuh Agents Active	**5/5**	`agent_control -l`
MITRE ATT&CK Techniques Covered	**8**	Detection matrix
Ansible Roles	**6**	site.yml play count
Automated Validation Checks	**17/17**	validate.sh
k3s Cluster Uptime	**75+ days**	`kubectl get nodes` AGE

4.8 Automated Validation Suite — 17/17 Passing

The platform includes `scripts/validate.sh`, an automated script performing 17 end-to-end health checks:

```
==========================================
   SOC Platform - Full Validation
   Sun May  4 2026
==========================================

--- Infrastructure ---
  [OK] k3s nodes ready (3/3)
  [OK] Wazuh pods running (4/4)
  [OK] TheHive pod running
  [OK] Suricata NIDS pods (2)

--- Agents ---
  [OK] Wazuh agents active (5)

--- Services ---
  [OK] Wazuh Dashboard
  [OK] TheHive
  [OK] Cortex
  [OK] Shuffle SOAR

```

```
--- Integration Tests ---
  [OK] Wazuh API authentication
  [OK] TheHive API accessible
  [OK] TheHive has cases (>= 1)
  [OK] TheHive soc@soc.local user
  [OK] Shuffle webhook
  [OK] Active Response configured
  [OK] Suricata NIDS (2 pods)
  [OK] custom-thehive integration script

=========================================
  PASSED: 17 / 17
  FAILED:  0 / 17
  STATUS: ALL SYSTEMS OPERATIONAL
=========================================
```

Listing 4.7: validate.sh output – 2026-05-04 (17/17)

4.8.1 Validation Check Descriptions

Table 4.7: validate.sh – all 17 check descriptions

#	Check	Command / API
1	k3s nodes ready (3/3)	`kubectl get nodes -no-headers | grep -c Ready`
2	Wazuh pods running (4/4)	`kubectl get pods -n wazuh | grep -c Running`
3	TheHive pod running	`kubectl get pods -n thehive | grep Running`
4	Suricata NIDS pods (2)	`kubectl get pods -n nids | grep -c Running`
5	Wazuh agents active (5)	`agent_control -l | grep -c Active`
6	Wazuh Dashboard reachable	`curl -sk https://172.16.10.9:32732`
7	TheHive reachable	`curl -s http://172.16.10.10:31000`
8	Cortex reachable	`curl -s http://172.16.10.10:9001`
9	Shuffle SOAR reachable	`curl -s http://172.16.10.10:3001`
10	Wazuh API authentication	`POST /security/user/authenticate`
11	TheHive API accessible	`POST /api/v1/query {listCase}`
12	TheHive has cases ($\geq$1)	JSON response length ≥ 1
13	TheHive soc@soc.local user exists	`GET /api/v1/user/soc@soc.local`
14	Shuffle webhook responds	`POST webhook {test:validate}`
15	Active Response configured	grep `firewall-drop` in ossec.conf
16	Suricata NIDS pods running	`kubectl get pods -n nids | grep Running`
17	custom-thehive script present	`ls /var/ossec/integrations/custom-thehive`

4.9 ISO 27001 and PCI-DSS Compliance Mapping

Table 4.8: Compliance mapping — ISO 27001 and PCI-DSS

Rule / Control	Description	ISO 27001 Control	PCI-DSS
Rule 100100	SSH brute force (5 failures/120 s)	A.9.4.2 Secure log-on	8.3.4
Rule 100101	Privilege escalation via sudo	A.9.4.4 Privileged utilities	10.2.5
Rule 5710	Failed SSH authentication	A.9.4.2 Secure log-on	8.3.4
Rule 92031	Suspicious process execution	A.12.4.1 Event logging	10.2.4
firewall-drop AR	Automatic IP blocking	A.13.1.1 Network controls	1.3.2
Wazuh FIM	File integrity monitoring	A.12.4.3 Log protection	10.5
Suricata NIDS	Network traffic inspection	A.13.1.2 Network security	1.1.2

General Conclusion and Perspectives

Summary of Achievements

This project successfully designed, implemented, and validated a fully operational cloud-native, open-source SOC platform for Flexos Tunisie. Starting from a security posture with no centralised monitoring, the project delivered:

- A **three-node k3s Kubernetes cluster** (v1.34.4+k3s1, Ubuntu 24.04) running 7 production workloads with 75+ days of uninterrupted uptime.
- **End-to-end automated detection and response**: SSH brute-force detected in **2.17 seconds**, attacker blocked in **under 4 seconds**—entirely without human intervention.
- **Four custom MITRE ATT&CK-mapped rules** (100100–100103) covering SSH brute force, privilege escalation, network reconnaissance, and Windows account discovery.
- **Full SOAR integration**: Wazuh alerts automatically trigger Shuffle workflows creating enriched TheHive cases; **847+ cases** have been generated.
- **Windows endpoint visibility**: Sysmon-enabled agent capturing 160+ events per day, with PowerShell execution and account discovery fully detected.
- **Network-layer detection**: Suricata processing **762,952+ alerts** across both worker nodes, with a Priority-1 finding identifying real unencrypted credential transmission.
- **Complete DevSecOps pipeline**: GitHub → GitHub Actions CI → Terraform IaC → AWX → Ansible 6-role playbook. Fully reproducible deployment demonstrated in a local KVM environment.
- **17/17 automated validation checks** passing, confirming all platform components are operational at every commit.

Objective Achievement Summary

Table 4.9: Objective achievement summary

Objective	Status	Evidence
Deploy Wazuh on Kubernetes (HA cluster)	**Achieved**	4 pods, 5 agents, 75+ days uptime
Integrate Suricata NIDS	**Achieved**	762k+ alerts, DaemonSet on 2 nodes
Automate incident response pipeline	**Achieved**	Shuffle + TheHive + Cortex, 847+ cases
Four custom MITRE-mapped detection rules	**Achieved**	Rules 100100–100103
Validate with real attack scenarios	**Achieved**	4 scenarios, 8 MITRE techniques
Full DevSecOps pipeline (AWX + Terraform)	**Achieved**	Job 41 successful, local demo running
17/17 automated validation checks	**Achieved**	validate.sh all passing
Zero licensing cost	**Achieved**	100% open-source stack

Known Limitations

1. **TheHive TLS:** Internal HTTP communication detected as Priority-1 by Suricata. TLS via cert-manager is the recommended remediation.
2. **Single-instance Docker services:** Cortex and Shuffle run as single-container Docker Compose services. A production deployment would use Kubernetes StatefulSets with HA backends.
3. **Windows agent version:** Agent 008 runs Wazuh v4.7.3 against manager v4.14.3. Compatible but an upgrade is recommended.
4. **CIS Windows benchmark:** The Windows endpoint scored 32% on the CIS SCA scan; full hardening (BitLocker, AppLocker) remains outstanding.
5. **ET SCAN rules:** The Emerging Threats scan-detection ruleset is not deployed; nmap scans appear as TCP flow events only, not as named alerts.

Future Perspectives

1. **Terraform Proxmox provider:** Automate production VM provisioning with `bpg/proxmox` Terraform provider, eliminating the last manual infrastructure step.

2. **Additional MITRE techniques:** Add detection rules for T1021 (lateral movement), T1558.003 (Kerberoasting), T1562.001 (defence evasion), and T1071 (application-layer C2).
3. **AWS cloud deployment:** Migrate the SOC stack to AWS EKS using the same Ansible roles, validating cloud portability and enabling hybrid-cloud monitoring.
4. **TheHive playbook automation:** Build Shuffle workflows that automatically assign cases to analysts, trigger Cortex analysis, and close resolved cases with evidence attached.
5. **MISP Threat Intelligence:** Connect MISP to Cortex for automated IoC lookups and community threat-intelligence sharing.
6. **Deception technology:** Deploy Cowrie SSH honeypots monitored by Wazuh to attract and study attacker techniques in a controlled environment.
7. **Full TLS everywhere:** Implement cert-manager with Let's Encrypt (or an internal CA) to enforce HTTPS across all platform APIs, resolving the Suricata Priority-1 finding.

Personal Reflection

This project was a demanding but deeply rewarding engineering experience. Deploying a production-grade security platform from scratch—navigating Kubernetes storage classes, Wazuh cluster networking, Sysmon rule tuning, Terraform VM provisioning, AWX pipeline integration, and CI/CD automation—required synthesising knowledge from across the computer science curriculum: operating systems, networking, distributed systems, software engineering, and cybersecurity. The measured KPIs (2.17 s detection, 4 s end-to-end containment, 847+ cases, 762k+ alerts, 75+ days uptime) validate that open-source tools, thoughtfully integrated and automated, can match commercial SOC capabilities at a fraction of the cost. This project demonstrates that enterprise-grade cybersecurity is not a privilege reserved for organisations with large security budgets.

Bibliography

[1] Wazuh, Inc. *Wazuh Documentation v4.14.3*, 2024. Consulté en mai 2026.

[2] TheHive Project / StrangeBee. *TheHive Documentation v5.2*, 2024. Consulté en mai 2026.

[3] TheHive Project / StrangeBee. *Cortex Documentation v3.1*, 2024. Consulté en mai 2026.

[4] Open Information Security Foundation (OISF). *Suricata Documentation v7.0*, 2024. Consulté en mai 2026.

[5] Shuffle AS. *Shuffle SOAR Documentation*, 2024. Consulté en mai 2026.

[6] Rancher Labs / SUSE. *k3s Lightweight Kubernetes Documentation*, 2024. Consulté en mai 2026.

[7] The Kubernetes Authors. *Kubernetes Documentation v1.34*, 2024. Consulté en mai 2026.

[8] Red Hat, Inc. *Ansible Documentation v2.17*, 2024. Consulté en mai 2026.

[9] dmacvicar. *Terraform Provider for libvirt*, 2024. Consulté en mai 2026.

[10] Red Hat, Inc. *AWX (Ansible Tower Open Source) Documentation*, 2024. Consulté en mai 2026.

[11] MITRE Corporation. *MITRE ATT&CK Enterprise Matrix v15*, 2024. Consulté en mai 2026.

[12] Docker, Inc. *Docker Documentation*, 2024. Consulté en mai 2026.

[13] Microsoft Corporation. *Sysmon v15 — System Monitor*, 2024. Consulté en mai 2026.

[14] Proxmox Server Solutions GmbH. *Proxmox VE Documentation v8*, 2024. Consulté en mai 2026.

[15] IBM Security. Cost of a data breach report 2024. Technical report, IBM Corporation, 2024.

[16] International Organization for Standardization. ISO/IEC 27001:2022 — information security management systems, 2022.

[17] PCI Security Standards Council. PCI DSS v4.0 — payment card industry data security standard, 2022.

[18] National Institute of Standards and Technology. NIST cybersecurity framework 2.0. Technical report, NIST, 2024.

[19] Proofpoint, Inc. *Emerging Threats Open Ruleset*, 2024. Consulté en mai 2026.

[20] Center for Internet Security. *CIS Microsoft Windows 10 Enterprise Benchmark v1.12.0*, 2023.

[21] Risto Vaarandi and Mauno Pihelgas. Using security logs for collecting and reporting technical security metrics. *Proceedings of the 2014 IEEE Military Communications Conference*, pages 294–299, 2014.

[22] Antoine Mohasseb et al. A comparative study of siem solutions for threat detection and compliance. *Journal of Network and Computer Applications*, 218:103706, 2023.

[23] Wazuh, Inc. *Wazuh Active Response Documentation*, 2024. Consulté en mai 2026.

Appendix A

validate.sh — Full Source Code

```
#!/bin/bash
export KUBECONFIG=/home/socadmin/.kube/config
THEHIVE_KEY="9O81h9pfpC7bBvSXh+S5gQ6/4mrULoBP"
THEHIVE_URL="http://172.16.10.10:31000"
PASS=0; FAIL=0

check() {
  local label="$1"; local cmd="$2"
  if eval "$cmd" &>/dev/null; then
    echo "  [OK] $label"; ((PASS++))
  else
    echo "  [FAIL] $label"; ((FAIL++))
  fi
}

echo "=========================================="; echo "   SOC Platform - Full Validation"
echo "   $(date)"; echo "=========================================="

echo ""; echo "--- Infrastructure ---"
check "k3s nodes ready (3/3)" \
  "[ \$(kubectl get nodes --no-headers | grep -c Ready) -eq 3 ]"
check "Wazuh pods running (4/4)" \
  "[ \$(kubectl get pods -n wazuh --no-headers | grep -c Running) -ge 4 ]"
check "TheHive pod running" \
  "kubectl get pods -n thehive --no-headers | grep -q Running"
check "Suricata NIDS pods (2)" \
  "[ \$(kubectl get pods -n nids --no-headers | grep -c Running) -ge 1 ]"

echo ""; echo "--- Agents ---"
check "Wazuh agents active (5)" \
  "[ \$(kubectl exec -n wazuh wazuh-manager-master-0 -- \
```

```
␣␣␣␣/var/ossec/bin/agent_control␣-l␣2>/dev/null␣|␣grep␣-c␣Active)␣-ge␣
    5␣]"

echo ""; echo "---␣Services␣---"
check "Wazuh␣Dashboard" \
  "S=\$(curl␣-sk␣-o␣/dev/null␣-w␣'%{http_code}'␣https
    ://172.16.10.9:32732);␣\
␣␣␣␣[␣\"\$S\"␣=␣200␣]␣||␣[␣\"\$S\"␣=␣302␣]"
check "TheHive" \
  "S=\$(curl␣-s␣-o␣/dev/null␣-w␣'%{http_code}'␣http
    ://172.16.10.10:31000);␣\
␣␣␣␣[␣\"\$S\"␣=␣200␣]␣||␣[␣\"\$S\"␣=␣302␣]"
check "Cortex" \
  "S=\$(curl␣-s␣-o␣/dev/null␣-w␣'%{http_code}'␣http
    ://172.16.10.10:9001);␣\
␣␣␣␣[␣\"\$S\"␣=␣200␣]␣||␣[␣\"\$S\"␣=␣303␣]"
check "Shuffle␣SOAR" \
  "S=\$(curl␣-s␣-o␣/dev/null␣-w␣'%{http_code}'␣http
    ://172.16.10.10:3001);␣\
␣␣␣␣[␣\"\$S\"␣=␣200␣]␣||␣[␣\"\$S\"␣=␣303␣]"

echo ""; echo "---␣Integration␣Tests␣---"
check "Wazuh␣API␣authentication" \
  "curl␣-sk␣-X␣POST␣https://172.16.10.9:30947/security/user/
    authenticate␣\
␣␣␣␣--user␣'wazuh-wui:MyS3cr37P450r.*-'␣|␣grep␣-q␣token"
check "TheHive␣API␣accessible" \
  "curl␣-sf␣-H␣'Authorization:␣Bearer␣$THEHIVE_KEY'␣\
␣␣␣␣-X␣POST␣'$THEHIVE_URL/api/v1/query'␣\
␣␣␣␣-H␣'Content-Type:␣application/json'␣\
␣␣␣␣-d␣'{\"query\":[{\"_name\":\"listCase\"}]}'␣|␣python3␣-c␣\
␣␣␣␣'import␣sys,json;␣json.load(sys.stdin)'"
check "TheHive␣has␣cases␣(>=␣1)" \
  "C=\$(curl␣-sf␣-H␣'Authorization:␣Bearer␣$THEHIVE_KEY'␣\
␣␣␣␣-X␣POST␣'$THEHIVE_URL/api/v1/query'␣\
␣␣␣␣-H␣'Content-Type:␣application/json'␣\
␣␣␣␣-d␣'{\"query\":[{\"_name\":\"listCase\"}]}'␣|␣\
␣␣␣␣python3␣-c␣'import␣sys,json;␣print(len(json.load(sys.stdin)))');␣\
␣␣␣␣[␣\"\$C\"␣-ge␣1␣]"
check "TheHive␣soc@soc.local␣user" \
  "S=\$(curl␣-s␣-o␣/dev/null␣-w␣'%{http_code}'␣\
␣␣␣␣http://172.16.10.10:31000/api/v1/user/soc@soc.local␣\
␣␣␣␣-u␣'admin@thehive.local:secret');␣[␣\"\$S\"␣=␣200␣]"
check "Shuffle␣webhook" \
  "curl␣-sf␣-X␣POST␣\
␣␣␣␣http://172.16.10.10:3001/api/v1/hooks/webhook_4030788a-6f3e-40c9-
    ab08-ff56836c96b1␣\
```

```
    -H 'Content-Type: application/json' \
    -d '{"test":"validate"}' | grep -q success"
check "Active Response configured" \
  "kubectl exec -n wazuh wazuh-manager-master-0 -- \
    grep -q 'firewall-drop' /var/ossec/etc/ossec.conf"
check "Suricata NIDS (2 pods)" \
  "[ \$(kubectl get pods -n nids --no-headers | grep -c Running) -ge 1 ]"
check "custom-thehive integration script" \
  "kubectl exec -n wazuh wazuh-manager-worker-0 -- \
    ls /var/ossec/integrations/custom-thehive"

echo ""; echo "========================================="
TOTAL=$((PASS+FAIL))
echo "  PASSED: $PASS / $TOTAL"; echo "  FAILED: $FAIL / $TOTAL"
[ "$FAIL" -eq 0 ] \
  && echo "  STATUS: ALL SYSTEMS OPERATIONAL" \
  || echo "  STATUS: $FAIL CHECK(S) FAILED"
echo "========================================="
```

Listing A.1: scripts/validate.sh – SOC platform 17-check validation script

Appendix B

MITRE ATT&CK Technique Details

ID	Name	Detection	Response
T1110.001	Brute Force: Password Guessing	Rule 100100 (freq 5/120 s, parent 5710)	firewall-drop 600 s
T1548.003	Abuse Elevation: sudo	Rule 100101 (level 14, match sudo bash)	firewall-drop 300 s
T1046	Network Service Discovery	Suricata flow events, Rule 100102	Logged, case created
T1087.001	Local Account Discovery	Sysmon EID 1 (net.exe), Rules 92039/100103	Logged
T1059.001	PowerShell	Sysmon EID 11, Rule 92205 (level 9)	Logged, analyst alerted
T1059.003	Windows Command Shell	Sysmon EID 1, Rule 92031	Logged
T1547	Autostart via Registry	Wazuh FIM registry rules 750–598	Case if level ≥ 10
T1078	Valid Accounts (HTTP creds)	Suricata SID 2006380, Priority 1	Logged, TLS advised

Appendix C

Platform Architecture Diagram Reference

Layer 1 — Physical/Hypervisor: One Proxmox 8.x host, VLAN `172.16.10.0/24`, five VMs (master, worker1, worker2, ubunttest, windows-soc).

Layer 2 — Kubernetes: k3s v1.34.4+k3s1, containerd 2.1.5-k3s1, Flannel CNI (`10.42.0.0/16`), Service network `10.43.0.0/16`, local-path StorageProvisioner, three namespaces (`wazuh`, `thehive`, `nids`).

Layer 3 — Security Stack:

- master: `wazuh-manager-master-0`, `wazuh-indexer-0`, `wazuh-dashboard`
- worker1: `wazuh-manager-worker-0`, `suricata-tkp6l`
- worker2: `thehive` pod, `suricata-vdvdg`, `cortex:9001` (Docker), `shuffle:3001` (Docker)

Layer 4 — Endpoints:

- Agent 005: worker2 (172.16.10.10) — Linux
- Agent 006: worker1 (172.16.10.5) — Linux
- Agent 007: ubunttest (172.16.10.11) — Ubuntu — attack target
- Agent 008: windows-soc (172.16.10.12) — Windows 10 + Sysmon v15

DevSecOps Pipeline:

- GitHub repository: `iiismailtriki/Cloud-native-open-source-SOC-platform--ESPRIT-PFE-`
- GitHub Actions: 3 jobs (lint-ansible, validate-yaml, security-scan)
- Terraform provider: `dmacvicar/libvirt` — 3 KVM VMs
- AWX: kind cluster at `http://localhost:8080`, Job 41 successful

- Ansible site.yml: 6 plays (common, k3s-master, k3s-worker, soc-tools-k8s, soc-tools-docker, wazuh-agent)

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454